Lab3 AttackLab 实验报告

1. 观察 input 函数的栈帧

input 是谁调用的？ `[#1] 0x401ad1 → main()`，即 main 调了 input。

前三条指令在干什么？ input 开头是：

```
push rbp        ; 把 main 的 rbp 保存下来
mov  rbp, rsp   ; rbp 指向新的栈帧底
sub  rsp, 0x10  ; 留出 16 字节给 expression[16]
```

$rbp+8 是什么？ 是返回地址 `0x401ad1`，对应 main 里 `call input` 后面的 `mov eax,0x0`。

这个值是谁写进去的？ 是 `call` 指令压的返回地址。

call、prologue、leave、ret 分别做什么？ `call` 往栈压入返回地址再跳转；prologue 是保存旧 rbp、建立新的栈帧；`leave`（`mov rsp,rbp; pop rbp`）释放局部变量、并恢复 rbp；`ret`（`pop rip`）弹出返回地址，结束一个函数。

2. 不同长度的输入会怎样

`expression[16]` 在 `[rbp-0x10]`，返回地址在 `[rbp+8]`，中间隔了 16（缓冲区）+ 8（保存的 rbp）= 24 字节。

输入 32 个 A，一共 33 字节，会将返回地址覆盖，ret 的时候地址非法，产生Segmentation fault。

3. 溢出执行 ./malware

payload 写成：

```
payload = b"./malware\x00" + b"A" * 14 + b"\x6b\x1a\x40\x00\x00\x00\x00\x00"
```

开头 `"./malware\x00"` 里的 `\0` 让 strcpy 只拷走 `"./malware"`；后面 14 个 A 加返回地址凑够 24 字节，把返回地址盖成 `0x401a6b`，程序一返回就去执行 `system("./malware")`。跑起来后 malware 发现自己父进程是 dark-calc。

4. main 函数的机器码

用 `x/16bx 0x401ac3` 看 main 入口的字节：

指令	机器码	字节数
push rbp	55	1
mov rbp, rsp	48 89 e5	3

指令	机器码	字节数
mov eax, 0x0	b8 00 00 00 00	5

5. 栈空间执行

dark-calc 这个程序栈地址是固定的且栈可以执行：

```
mov edi, 4        ; bf 04 00 00 00
mov eax, 0x401a87 ; b8 87 1a 40 00   grade_eval 的地址
call rax          ; ff d0
```

因为 `grade_eval` 要参数等于 4 才会打印，所以先 `mov edi,4` 再把它的地址 `0x401a87` 放进 rax 调用。

6. ROP

在 `__libc_csu_init` 里 `pop r15`，机器码是 `41 5f`。如果从 `5f` 开始，即为 `pop rdi`，后面 `c3`（ret），这样就有了 `pop rdi; ret`，地址是 `0x401a33`。

```
24 字节填充 + pop rdi;ret(0x401a33) + 4 + grade_eval(0x401978)
```

执行时 `pop rdi` 把栈上的 4 弹进 rdi，`ret` 跳跳转 `grade_eval(0x401978)`

7. 绕过 Canary

结构体 `log_frame`，字段顺序是 `expression[128]`、`logMsg[128]`、`len`、`pad`。memcpy 的目标是由 `f.len` 决定的。用 gets 溢出把 `f.len`（在 `rbp-0x20`）改成 168，这样目标就变成 `f.logMsg + 168 = rbp + 8`，返回地址。

思考题：把 len 这类字段放到溢出够不着的位置，例如放在 canary 和返回地址之间，让攻击者无法先改它。